PRACTICAL – 4

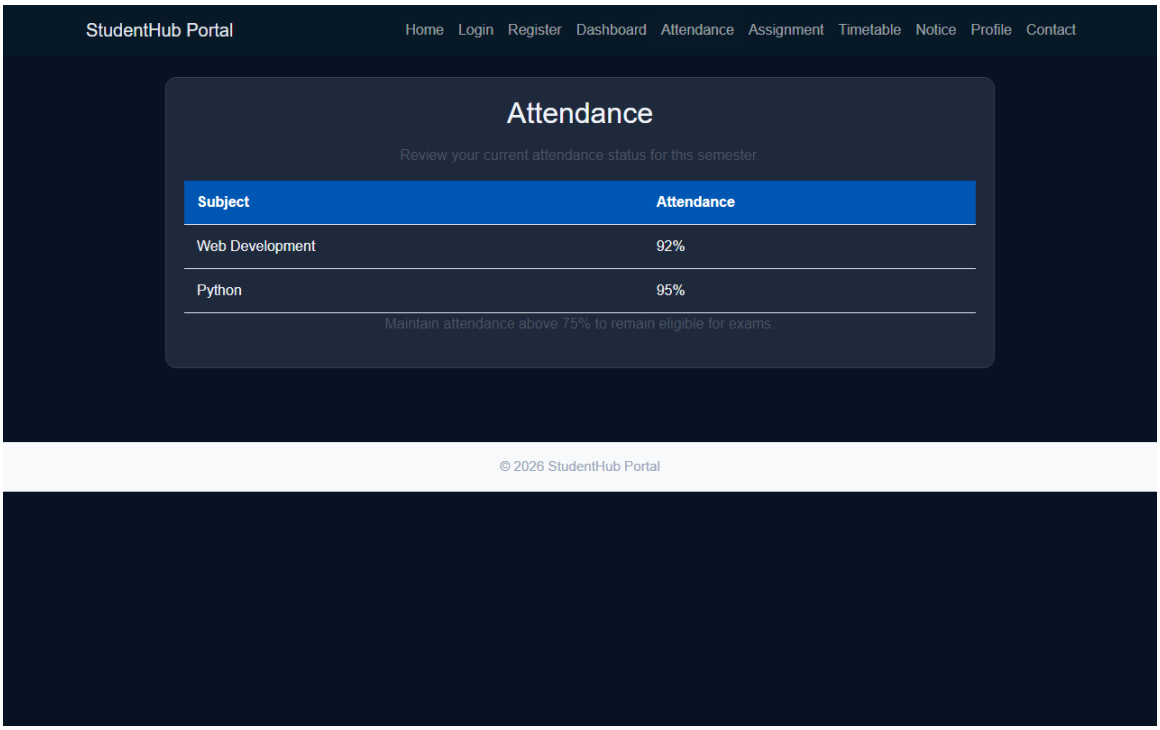
Problem Definition: Add dynamic UI components such as collapsible FAQ, modal popup, image/content slider, notification banner, hamburger menu, and light/dark theme switcher using JavaScript.

Output:

Home Page with Theme Switcher & Dynamic Interactivity:



Interactive Attendance Status View:



Question & Answer:

1. How are DOM elements selected and modified?

Ans. DOM elements are selected using modern JavaScript methods such as document.getElementById(), document.querySelector(), and document.querySelectorAll(). Once selected, elements are modified by updating properties like textContent, innerHTML, modifying inline styles, or toggling classes via classList.add(), classList.remove(), and classList.toggle(). This enables dynamic client-side content updates without reloading the webpage.

2. Are event listeners attached correctly?

Ans. Yes. Event listeners are bound cleanly using addEventListener() for user interaction events including click, input, change, and submit. Event delegation and decoupling JavaScript event handling from HTML attributes improves code maintainability, prevents memory leaks, and adheres to modern web standards.

3. Is localStorage used for remembering theme choice?

Ans. Yes. localStorage is used to persist the user's selected theme (light or dark mode) and customized heading preferences. When the theme toggle button is clicked, the selection is saved using localStorage.setItem('theme', ...). Upon subsequent page loads, the preference is retrieved via localStorage.getItem('theme') and applied automatically.

4. Does interactivity improve usability without breaking accessibility?

Ans. Yes. Interactive UI elements improve usability by providing immediate visual feedback and collapsible content. Accessibility is preserved by using proper semantic

HTML5 tags, visible focus indicators, ARIA attributes (such as aria-expanded and aria-live), and keyboard-friendly navigation for interactive controls.

Conclusion / Learning Outcome:

In this practical, interactive client-side functionality was successfully implemented on the StudentHub portal using vanilla JavaScript ES6+. DOM selection methods and event listeners were implemented for theme toggling, interactive banners, and heading customizations. Data persistence across sessions was achieved using the Web Storage API (localStorage). The practical reinforced the fundamentals of event handling, DOM tree manipulation, state management, and accessible UI interactions.

Github link :- https://github.com/07vish/sh.git